Figma MCP ハンズオン準備：追加リソース & 優先度見直し

> 前回資料（`figma-mcp-handson-prep.md`）の補足。最新情報の網羅的調査の結果、**前回よりも優先すべき情報源**と**実践的な知見**が判明した

---

0. 結論：前回の優先順位の見直し

前回の最優先★★★（公式ヘルプ + 公式ブログ）は基礎として有効だが、**「明日から動けるか」という観点では実践記事のほうが上**。以下が今回の最優先。

🔥 最優先で読むべき記事（前回資料を上書き）

優先	記事	判定理由
★★★★★	**Figma MCP × GitHub CopilotでReact UI実装してわかった実践Tips（NTTデータ Zenn）**	**今回の目的（Copilot + Next.js + 月曜ハンズオン）に完全一致**。Custom instructions + Prompt files の構成が載っている
★★★★★	**Figma MCPを活かすためのデータ作りとプロンプト 20パターンの検証（Qiita）**	**20通りの組み合わせで実証済み**。Variables は逆効果になる場合があるなど、常識を覆す結果が多数
★★★★★	**「8割自動化」できた！FigmaMCPを使った超時短実装フロー（noco株式会社）**	`.github/instructions/` 配下の `html.instructions.md` / `css.instructions.md` の**実物サンプルが丸ごと公開**されている
★★★★	Figma MCPを攻略して最高のコード生成を実現する（Zenn）	失敗例 → 改善の全過程。Playwright MCP による視覚検証も解説
★★★★	Figma MCP でデザインシステムを提供して AI に一貫したフロントエンドコードを書かせる（azukiazusa.dev）	Next.js + Tailwind の完全な実装例。Storybook MCP との併用例も

重要な方針転換：前回「一次情報優先」と書いたが、**Figma MCP は挙動の個体差が大きく、実証ベースの記事のほうが学習効率が高い**。公式ドキュメントは辞書的に使う。

---

1. 前回資料で言及できていなかった決定的事項

1.1 Code Connect の重要性（最重要）

Figma 公式ドキュメントの以下の一文が、MCP 運用の核心：

> **"This is the #1 way to get consistent component reuse in code. Without it, the model is guessing."** （既存コンポーネントを一貫して再利用させるための唯一無二の方法。これなしではモデルは推測しているだけ）

Code Connect がない場合、AI は毎回「新しい Button コンポーネント」をゼロから生成する。既存の `src/components/ui/Button.tsx` があっても、それを使わない。**今回のプロジェクトで Code Connect が設定されているかは、金曜のうちに確認すべき。**

確認方法： 1. Figma でコンポーネントを選択 2. Dev Mode で右側のコード欄に「独自コードスニペット」が出ていれば Code Connect 済み 3. 出ていなければ未設定 → instructions で明示的にコンポーネント場所を伝える必要がある

1.2 Prescriptive vs Descriptive 出力問題

公式 Figma MCP は **prescriptive（処方的）** に React + Tailwind コードを出力する。これが**コンテキストを汚染する**：

```
問題: AI が Figma が出した自動生成コード構造を真似してしまい、
      プロジェクトの既存パターンに従わない
結果: 「AI出力をリファクタリングする」という本末転倒な作業が発生
```

対策：プロンプトで明示的に「プロジェクトの既存パターンを使え」「Figma MCP の出力はあくまで参考」と指示する。

1.3 Variables が逆効果になる場合がある（20パターン検証の結論）

Qiita の20パターン検証による衝撃的な発見：

設定	結果
Variables のみ有効	**Variables が他要素の効果を打ち消す**（悪化）
Auto Layout + Layer Names + Annotations	**最高品質**
全要素有効	中程度（Variables ありより悪い場合も）

教訓：Figma の Variables をフル活用すれば良いと単純に思っていたが、**プロンプトで明示的に「Variables を標準 Tailwind クラスに変換せよ」と指示しないと、`var(--xxx)` のまま出力される。**

推奨プロンプト指示：

```
Map Figma Variables to standard Tailwind classes:
- Color variables → bg-slate-50, text-blue-600
- Spacing variables → p-4, gap-6
- Typography variables → text-sm, font-medium
- Avoid CSS variables like var(--slate-50)
```

1.4 チャット単位の修正：1画面 ≠ 1チャット

前回「1機能=1チャット」と書いたが、実践記事から学んだ**より正確な粒度**：

```
❌ 1画面を丸ごと実装依頼 → 精度が大幅に低下
✅ 1セクション単位で分割 → 3〜4分×4回で1ページ完成
✅✅ コンポーネント単位 → 既存定義を活用でき最も安定
```

noco社の事例では、1ページを4セクションに分けて実装。**前回資料の「1機能=1チャット」は粗すぎた**。

1.5 デザインとコードどちらを正とするかの運用設計

これは実装手順と同じくらい重要な論点で、前回資料には抜けていた：

開発スタイル	正とするもの	運用
ウォーターフォール	Figma デザイン	デザイン完成後に実装、デザインを絶対とする
アジャイル	ソースコード	実装しながら仕様変更、必要に応じて Figma を更新

Figma MCP は**コード → Figma への自動更新はできない**。双方向性は限定的。この前提をハンズオン参加者に説明できるとプロに見える。

---

2. instructions / prompt files の実物サンプル

2.1 noco株式会社が公開している html.instructions.md（抜粋）

前回資料では「書くべき項目」しか書けなかったが、実物が公開されている：

```
## HTML整形の指示書
目的：FigmaのMCPを使って、Figmaデータを再現したい。
     既存のサービスサイトで使われている記法に則ったマークアップに揃えたい。

### 要件定義
1. 1階層目のdivは section要素にしてください
2. 不要なdivは削除してください
3. divの入れ子が多い場合は、削除して可能な限りシンプルな階層構造にしてください
4. ランダムに生成された、意味のないクラス名は削除してください。
   代わりに、文章構造的に適切なクラス名を追加してください
5. srcは src="" として中を空にしてください
6. altは alt="" として中を空にしてください
7. section要素には section、h1要素には heading-1、h2要素には heading-2 と large、
   h3要素には heading-3、h4要素には heading-4、p要素には paragraph と large の
   class名をつけてください
8. Figma MCPで取得したAnnotationを読み取って参考にしてください
9. 実装後、エラーがないか必ず確認してください

### 禁止事項
- HTMLの中に入っている文章のテキストを変更しない
- shadcn/uiなどのJSXベースUIコンポーネントは使用しない
- Figmaのレイヤー構成を勝手に変えない
```

2.2 Figma 公式 GitHub の Copilot instructions 推奨テンプレート

`figma/mcp-server-guide` リポジトリから：

```
## Figma MCP Integration Rules

### Required flow (do not skip)
1. Run get_design_context first to fetch the structured representation
2. If the response is too large or truncated,
   run get_metadata to get the high-level node map
   and then re-fetch only the required node(s) with get_design_context
3. Run get_screenshot for a visual reference
4. Only after you have both get_design_context and get_screenshot,
   download any assets needed and start implementation
5. Translate the output (usually React + Tailwind) into this project's
   conventions, styles and framework
6. Validate against Figma for 1:1 look and behavior before marking complete

### Implementation rules
- Treat the Figma MCP output (React + Tailwind) as a representation of design
  and behavior, not as final code style
- Replace Tailwind utility classes with the project's preferred utilities
- Reuse existing components instead of duplicating functionality
- Use the project's color system, typography scale, and spacing tokens consistently
- Respect existing routing, state management, and data-fetch patterns
```

この公式テンプレートを叩き台に、Next.js プロジェクト用にカスタマイズするのが最短ルート。

3. 追加の強力なワークフロー

3.1 Playwright MCP による視覚検証ループ

Figma MCP で生成したコードを Playwright MCP で撮影し、**Figma と実装を視覚比較する自動検証ループ**が構築できる：

```
Figma MCP で生成
  ↓
Next.js アプリ起動
  ↓
Playwright MCP でスクリーンショット撮影
  ↓
AI が Figma デザインと比較 → 差分指摘
  ↓
修正プロンプト生成 → 再実装
```

ハンズオン月曜には間に合わないが、**「こういう発展形がある」と話せるだけで深みが出る**。

3.2 Storybook MCP との組み合わせ

azukiazusa 氏の記事：

```
Figma MCP → コンポーネント取得
  +
Storybook MCP → 既存コンポーネントの Story を参照
  ↓
AI が既存コンポーネントを再利用して実装
```

Code Connect の簡易版として機能する。もしプロジェクトに Storybook があれば、Code Connect を設定しなくてもある程度の再利用性が担保できる。

3.3 モデル選択の実証結果

複数の記事で共通する結論：

モデル	評価
Claude Sonnet 4 / 4.5	画面再現の最優秀、日本語プロンプトへの追従性も高い
GPT-4o	論理構造は良いが、視覚再現度で劣る
Gemini Pro	ベストプラクティス遵守は強いが、細部が弱い

Copilot Chat でモデルを選べるなら Claude 系を選ぶべき。

4. 想定質問への追加回答

前回の想定 Q&A に加えて、以下も準備しておくと強い：

質問	回答の方向性
「なぜ Variables を使っているのに CSS 変数で出力される？」	プロンプトで明示的に「標準 Tailwind クラスに変換」を指示する必要がある
「生成コードが既存コンポーネントを使わない」	Code Connect 未設定、または instructions で場所を明示していない
「1ページを丸ごと生成したら品質が落ちた」	1セクション単位、理想はコンポーネント単位で分割
「Figma を修正したらコードも自動更新できる？」	双方向同期はできない。運用設計でどちらを正とするか決める必要がある
「どのモデルを使うべき？」	現時点では Claude Sonnet 4 系が最優秀
「Code Connect はハンズオンで教えるべき？」	概念だけ紹介。設定は別セッションが望ましい

---

5. 見つけた追加の一次情報（優先度中）

日本語で読める実証記事

情報源	特徴
Sky Style Blog: Dev Mode MCP Server 検証	「既存実装を参考にする」指示の効果検証
スネークぴょんぴょん: VS Code 導入手順	2026年3月の双方向ワークフロー解説
ThunderMiracle's Blog: Cursor での運用	SP→PC のレスポンシブ化フロー

英語の重要記事

情報源	特徴
Figma MCP: The CTO Guide to Design-to-Code in 2026	プロダクション投入の全側面。セキュリティ、コスト管理含む
Bridging Design and Code（Michel Velis）	Code Connect 統合の実装例
figma/mcp-server-guide（GitHub 公式）	**公式のルールサンプルを含む**

---

6. 修正された土日計画

土曜 AM：Figma の整備度 × プロンプトの効果を実証

前回「対象機能を選ぶ」と書いたが、Qiita の20パターン検証を踏まえて、以下の**比較実験**を加えると理解が深まる：

```
同じフレームに対して3パターンのプロンプトで生成
  A. 「このデザインを実装して」（最小プロンプト）
  B. instructions あり + 一般的な指示
  C. instructions あり + Variables変換/セマンティックHTML明示
  ↓
出力を比較 → プロンプトの威力を体感
```

これをハンズオンのデモに組み込めば、**「なぜ instructions とプロンプトが重要か」を言葉ではなく見せられる**。

日曜 PM：ハンズオン資料に「失敗例」セクションを入れる

初学者が必ず遭遇する失敗を、意図的にデモする：

1. 「1ページ丸ごと」を試して品質低下を見せる
2. Variables だけ有効で `var(--xxx)` 出力される失敗を見せる
3. instructions なしで既存コンポーネントを使わない失敗を見せる
4. それぞれの**修正プロセス**を見せる

成功例だけ見せるハンズオンは記憶に残らない。失敗と修正のセットが最強。

---

7. 一言で

前回資料の骨格は正しいが、**「公式ドキュメント優先」は修正が必要**。日本語実証記事3本（NTTデータ / Qiita20パターン / noco）が**今回の目的に対しては公式よりも有用**。

特に **Qiitaの20パターン検証**は、「Variables を使えばいい」という素朴な思い込みを覆す内容で、ハンズオンで語れると一気に信頼性が上がる。